

Rodrigo Yupanqui Rubio

Ingeniería Electrónica · Sistemas embebidos, IoT e instrumentación

✉ rodrigo.yupanqui@pucp.edu.pe [in linkedin.com/in/rodrigo-yupanqui](https://www.linkedin.com/in/rodrigo-yupanqui) github.com/RodrigoYupanqui 📍 Lima, Perú

Perfil

Estudiante de Ingeniería Electrónica en la **Pontificia Universidad Católica del Perú (PUCP)**, ubicado en el **5 % superior de Ingeniería Electrónica** y en el **10 % superior de la Facultad de Ciencias e Ingeniería**. Practicante de Ingeniería Electrónica en **DIACSA** (sensores, firmware, instrumentación, adquisición de datos e integración de hardware). Transformo requisitos de ingeniería en prototipos funcionales combinando electrónica, firmware, integración mecánica y procesamiento de datos.

Capacidades técnicas

Firmware	C/C++, Arduino-ESP32 (FreeRTOS, ISR, temporizadores), Python
Plataformas	ESP32, ESP32-S3, STM32, Arduino, Raspberry Pi 4B/5
Conectividad	LoRa (SX1278), Wi-Fi, WebSocket seguro (WSS), UART, I ² C, SPI, I ² S
Sensórica	IMU, magnetómetro, GPS, BME280, INA219, LiDAR, cámaras, encoders
Control	Regulación PI con <i>anti-windup</i> , filtrado EMA, PWM, servos
Visión / IA	OpenCV, YOLOv11n en el borde, segmentación HSV, nubes de puntos
Diseño HW	EasyEDA, LTspice, Proteus, Inventor, AutoCAD, PCB a medida
Software	FastAPI, Next.js, Git/GitHub, Linux, MATLAB, NumPy/Pandas

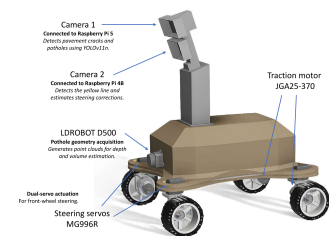
Experiencia y liderazgo

- **DIACSA** — Practicante de Ingeniería Electrónica.
- **IEEE Computer Society PUCP** — Director de Proyectos; liderazgo de equipos multidisciplinarios.
- **Huawei Seeds for the Future** — participante seleccionado.
- **Siemens, Programa Regional de Eficiencia Energética** — análisis de optimización energética.
- Instructor voluntario en talleres de electrónica.
- **Co-autor** del manuscrito sobre CrackScan (PUCP, Laboratorio de Procesamiento Digital de Señales).
- Finalista (17 equipos) en la **Primera Competencia Nacional de Pequeños Satélites 2024**.

Mapa de capacidades IoT demostradas en los proyectos

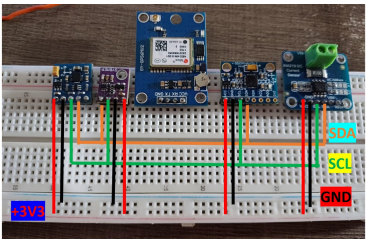
Capacidad	1 CrackScan	2 CubeSat	3 Arpeggiator
Adquisición multisensor	Encoders, LiDAR, cámaras, GPS	6 sensores en bus I ² C + UART	13 entradas, 2 ADC
Comunicación inalámbrica	WSS/JSON, telemetría a 1 Hz	LoRa 433 MHz, paquetes de 50 car.	—
Comunicación entre nodos	UART 115 200 baud con <i>timeout</i>	Enlace TX/RX ESP32-ESP32	I ² S a DAC
Control en lazo cerrado	PI de velocidad, lazo de 100 ms	Despliegue por altitud con histéresis	Filtro IIR 1.er orden
Edge AI / procesamiento	YOLOv11n en RPi 5, RANSAC	Altitud barométrica	Síntesis por LUT
Diseño de PCB y mecánica	PCB propia, chasis 4 ruedas	2 PCB (EasyEDA) en estructura 1U	PCB doble capa, MDF
Interfaz de monitoreo	Dashboard web (PWA)	GUI de estación terrena	LEDs WS2812B

Proyectos destacados



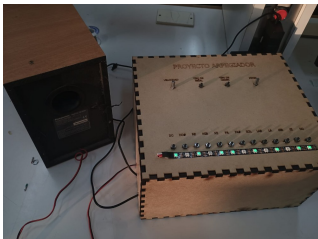
01 CrackScan

Rover autónomo con YOLOv11n, LiDAR y navegación visual; control distribuido en 3 nodos. *Co-autor de manuscrito.*



02 CubeSat 1U

Telemetría LoRa de 21 variables, 6 sensores y despliegue de paracaídas por altitud. *Finalista nacional (17 equipos).*



03 Arpeggiator

Instrumento musical ESP32-S3 con audio I²S, PCB propia y gabinete en MDF. *De la idea al producto.*

Qué aporte a un proyecto IoT

- **Capa de dispositivo:** selección de componentes, esquemáticos, PCB y firmware robusto (ISR, temporizadores, *timeouts* de seguridad).
- **Capa de conectividad:** LoRa, Wi-Fi, WSS, UART, I²C/SPI/I²S; protocolos propios documentados.
- **Capa de datos y nube:** telemetría JSON, servidor FastAPI, dashboard web y estaciones de monitoreo.
- **Edge AI y validación:** inferencia en el borde, métricas reportadas (mAP, F1, FPS) y documentación técnica rigurosa.

CrackScan — Rover autónomo de inspección de pavimentos

Visión por computador · LiDAR · control embebido distribuido | Proyecto Electrónico 1 (2025-1), PUCP | Asesor: S. E. Romero

0.78

mAP@0.5 (YOLOv11n)

6.10 FPS

inferencia en RPi5 (163.8 ms)

7123 pts

nube LiDAR en 0.40 s

0 %

intervención manual (pista evaluada)

Descripción

Rover de bajo costo (componentes COTS) que **detecta grietas y baches**, **reconstruye en 3D los baches detectados** y **navega siguiendo una línea amarilla**. Arquitectura distribuida de tres nodos:

- **Raspberry Pi 5:** YOLOv11n (grietas/baches), activación *selectiva* del LiDAR LDROBOT D500, volumen del bache, servidor FastAPI.
- **Raspberry Pi 4B:** segmentación HSV de la línea, ROI trapezoidal, centroide ponderado y ángulo de corrección a ≈ 20 Hz.
- **ESP32-WROOM-32U:** PI de velocidad de 2 motores, 2 servos de dirección sincronizados, WSS y telemetría.

Detalles técnicos

Control de motores (cada 100 ms, $T_s = 0.1$ s, 1050 pulsos/vuelta):

$$\omega[k] = \frac{600 n_{\text{enc}}[k]}{1050} \text{ RPM}, \quad e[k] = \omega^* - \omega[k] \quad (1)$$

$$I[k] = \text{sat}_{[0,255]}(I[k-1] + K_i e[k] T_s) \quad (2)$$

$$u[k] = \text{sat}_{[0,255]}(K_p e[k] + I[k]) \quad (3)$$

con $K_p = 1.8$, $K_i = 3.2$ y *anti-windup* por saturación del integrador.

Dirección autónoma: ángulo recibido por UART (A: <ang>, V: <ok>), recortado a $\pm 45^\circ$ y suavizado con EMA:

$$\theta_f[k] = (1 - \alpha) \theta_f[k-1] + \alpha \theta[k], \quad \alpha = 0.25 \quad (4)$$

con banda muerta de 1° , servos limitados a 60° – 120° y **parada de seguridad** si el enlace UART calla > 200 ms.

Volumen del bache: plano de referencia por RANSAC y mapa de profundidad d_{ij} en rejilla regular: $V = \sum_{i,j} d_{ij} \Delta A_{ij}$.

Clase	Precisión / Recall / F1	FN
Grietas	0.84 / 0.87 / 0.85	83
Baches	0.71 / 0.76 / 0.74	186

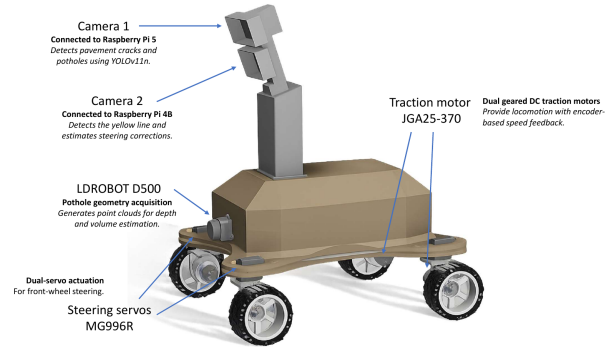
Entrenado con 7244 imágenes; prueba con 265 imágenes (640×480).

Interfaces y protocolos

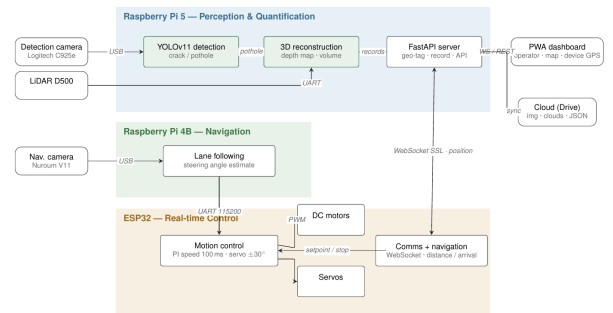
RPi4B→ESP32	UART 115200 baud, 8N1: A: -21.20, V: 1; START_AUTO/STOP_AUTO en sentido inverso.
Backend↔ESP32	WSS (443, /ws) con JSON: control, navigation, navigation_target; telemetría sensor a ≈ 1 Hz.
Seguridad	Credenciales fuera del repositorio (secrets.h); parada a velocidad cero ante pérdida de enlace.

Resultados y análisis crítico

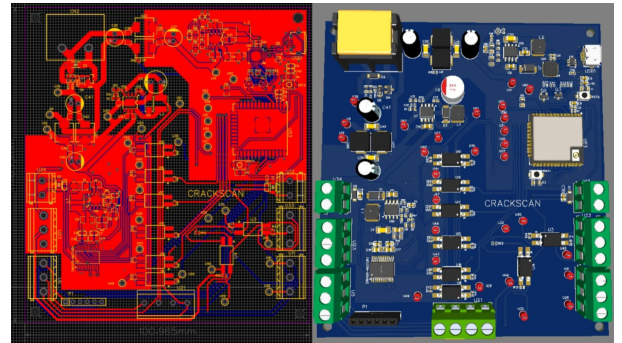
- Navegación continua con detección válida de la línea en la pista evaluada, **sin intervención manual**.
- Los **falsos negativos de baches (186)** son el modo de fallo crítico: inhiben el LiDAR selectivo. La validación del volumen contra una medición física queda como trabajo siguiente.



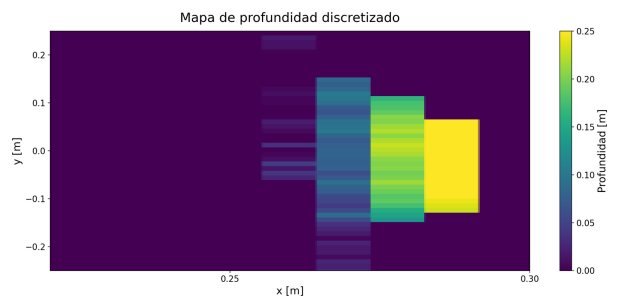
Plataforma: cámaras, LiDAR D500, tracción JGA25-370 y dirección con 2 servos MG996R.



Arquitectura de software distribuida (RPi5, RPi4B, ESP32).



PCB propia de integración electrónica.



Mapa de profundidad LiDAR: bache de 1348.73 cm^3 en 0.0111 m^2 .

Integración de **hardware electrónico**, arquitectura embebida y **comunicación entre módulos** (UART, WSS/JSON), integración de sensores y actuadores, y la **interfaz web de monitoreo**. Co-autor del manuscrito (ESP32 publicado en GitHub: 8 módulos de firmware, protocolos y hardware documentados).

CubeSat 1U — Monitoreo ambiental y telemetría LoRa

Equipo *Arquitectos Galácticos* | 1.ª Competencia Nacional de Pequeños Satélites 2024 (INRAS-PUCP, VRI-PUCP, CONIDA)

17

equipos finalistas

100 m

caída libre desde dron

6 sensores

I²C + UART, 21 variables

2 PCB

diseñadas en EasyEDA

Descripción

Satélite 1U lanzado desde un dron a 100 m y liberado en caída libre. Un **ESP32** adquiere temperatura, presión, humedad, aceleración, giro, campo magnético, tensión/corriente y posición GPS; los empaqueta en **JSON** y los transmite por **LoRa (SX1278, 433 MHz)** a una estación terrena con interfaz gráfica en tiempo real. Al descender, el sistema **libera el paracaídas** mediante dos servos SG90.

Detalles técnicos

Altitud relativa desde el BME280 (fórmula barométrica internacional, referenciada al suelo de arranque h_0):

$$h = \frac{1 - (P/1013.25)^{1/5.2559}}{2.25577 \times 10^{-5}} - h_0 \quad [\text{m}], P \text{ en hPa} \quad (5)$$

Lógica de despliegue con histéresis (evita disparos por ruido): el sistema se *arma* al superar 85 m y *despliega* (servos a 90°) cuando, ya armado, desciende por debajo de 80 m:

$$\text{armado} \leftarrow \text{armado} \vee (h > 85) \quad (6)$$

$$\text{despliegue} = \text{armado} \wedge (h < 80) \quad (7)$$

Paquetización LoRa: el JSON se fragmenta en $N = \lceil L/50 \rceil$ paquetes de hasta 50 caracteres con 50 ms entre paquetes; el receptor reensambla y extrae cada variable con `sscanf`.

Módulo	Función	Bus
ESP32 DevKit V1	MCU dual-core 240 MHz, 520 KB SRAM	—
BME280	Presión, temperatura, humedad (−40..85 °C)	I ² C
MPU6050	Acel. ±16 g + giroscopio, ADC 16 bits	I ² C
QMC5883L	Magnetómetro 3 ejes (±8 G), calibrado	I ² C
INA219	Corriente/tensión (resol. 0.8 mA / 4 mV)	I ² C
NEO-6M	GPS, 50 canales, −161 dBm	UART
SX1278 (Ra-02)	LoRa +20 dBm, hasta −148 dBm	SPI

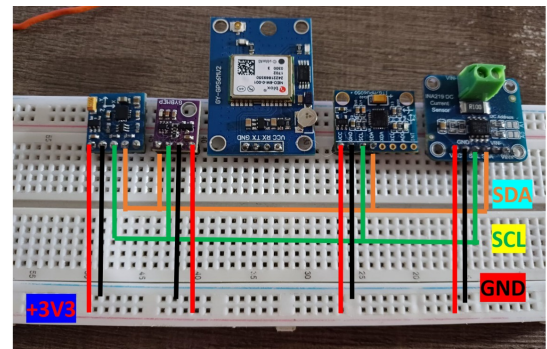
Calibración y pruebas

Magnetómetro: corrección de *offset* y escala por eje, $m_c = s \odot (m - b)$, con $b = (-276, 269, -348)$ y $s = (1.08, 1.11, 0.86)$; el acimut se obtiene de m_c .

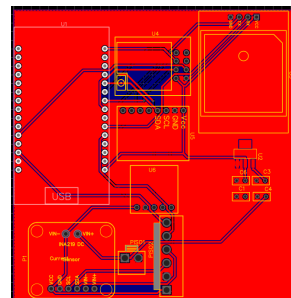
- Pausa de 500 ms entre ciclos de adquisición y transmisión, con reintentos de inicialización de cada sensor.
- Prueba de telemetría con antenas helicoidales y pruebas térmicas en el INRAS, visualizadas en tiempo real en la GUI de estación terrena.



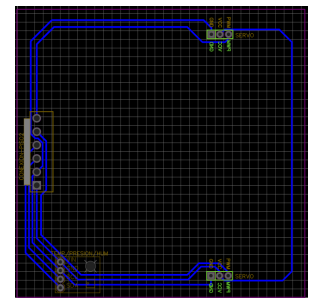
Fase final de la competencia (CNPS 2024).



Validación en protoboard: bus I²C común (SDA/SCL/3V3/GND) y GPS.



PCB piso 2: ESP32, GPS, LoRa, IMU.



PCB piso 3: BME280 y servos.

Selección de componentes (matriz morfológica, diagrama de flujo, criterios de precio/disponibilidad/viabilidad), **firmware en C++ para ESP32** (adquisición de todos los sensores y transmisión LoRa), **esquemáticos y PCB** en EasyEDA respetando las dimensiones del CubeSat. *Lección aprendida:* para una siguiente iteración, baterías Li-Ion de 3.7 V en lugar de pilas alcalinas y un mecanismo de despliegue más robusto.

ESP32-S3 Arpeggiator — Instrumento musical digital autónomo

Síntesis digital · audio I²S · PCB y gabinete propios | Diseño Electrónico 2 (IIEE17, 2024-2), PUCP | Asesor: H. Pratt

03

13

notas (C4–C5), 3 ondas, 3 modos

16 bit

I²S a 44.1 kHz

+15

iteraciones de firmware y hardware

13 LED

WS2812B de realimentación

Descripción

Instrumento independiente basado en un **ESP32-S3**: el usuario selecciona cualquier combinación de notas en un teclado de una octava y el equipo las reproduce como arpeggio *ascendente*, *descendente* o *aleatorio*, con onda senoidal, cuadrada o triangular, velocidad y tono ajustables por potenciómetro. El audio sale por I²S hacia un DAC PCM5102 y un amplificador clase D TPA3116D2.

Detalles técnicos

Cadena de señal:



Afinación con tono continuo $p \in [-0.5, 0.5]$ (potenciómetro, ADC de 12 bits), y **duración** repartida entre las notas activas:

$$f_{aj} = f_0 \cdot 2^p, \quad T_{nota} = \frac{T_{ciclo}}{N_{activas}} \quad (8)$$

Oscilador por tabla (LUT): 256 muestras por forma de onda ($x[m] = 32767 \sin(2\pi m/256)$ para la senoidal), con $f_s = 44.1$ kHz y bloques de 64 muestras:

$$N_p = \left\lfloor \frac{f_s}{f_{aj}} \right\rfloor, \quad i[n] = \left\lfloor \frac{(n \bmod N_p) \cdot 256}{N_p} \right\rfloor \quad (9)$$

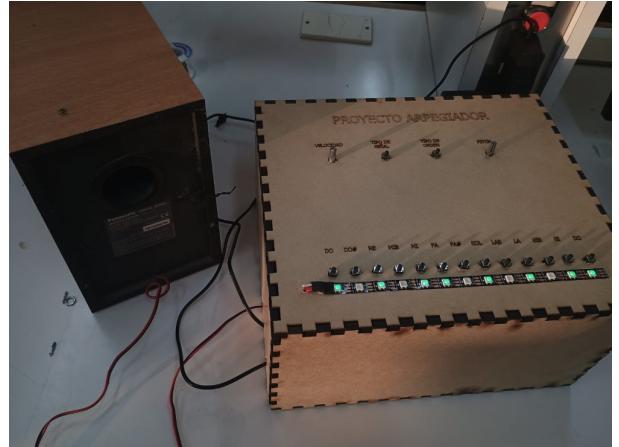
Filtro de suavizado de primer orden (estado persistente entre bloques y notas), con coeficiente $a \in [0, 1]$:

$$y[n] = y[n-1] + a(x[n] - y[n-1]) \quad (10)$$

Subsistema	Implementación	GPIO
Audio I ² S	BCLK / LRCLK / DATA → PCM5102	4 / 5 / 18
Notas (C4–C5)	13 pulsadores con <i>toggle</i> y LED por nota	8, 12–17, 19–21, 35–37
Controles	Velocidad y tono (ADC), modo y onda (botones)	3, 10, 40, 39
LEDs	Cadena de 13 WS2812B (un hilo)	38

Lecciones y mejoras previstas

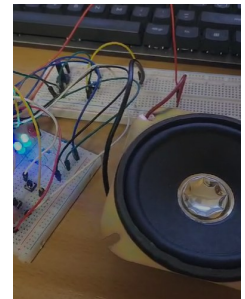
- Hardware**: la huella del ESP32-S3 en la PCB no encajó como se esperaba y el gabinete resultó más alto de lo necesario; una 2.ª revisión corrige huellas, orificios de montaje y cableado de pulsadores.
- Firmware**: reproducción bloqueante y `delay()` como antirrebote → máquina de estados con `millis()`, audio por DMA/tarea FreeRTOS, DDS con acumulador de fase y envolvente ADSR.



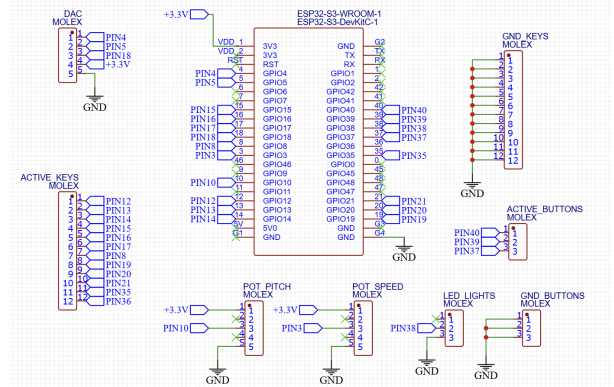
Prototipo final: gabinete MDF de 5.5 mm (≈25×30×15 cm), LEDs de nota y altavoz.



PCB doble capa fabricada.



Prueba DAC + amplificador.



Esquema: ESP32-S3 y conectores Molex.

Integrante del equipo de 4 estudiantes (M. Bruschi, R. Yupanqui, E. Torres, J. López). Flujo completo **idea** → **prototipo** → **producto**: prueba de DAC con senoidal de 400 Hz, prototipo PWM, migración a I²S, **diagnóstico y eliminación del ruido intermitente (popcorn)** reescribiendo la generación de muestras, PCB propia y gabinete cortado en láser. Repositorio con **revisión de código** documentada: límites conocidos (reproducción bloqueante, `delay()` como antirrebote) y plan de mejora (acumulador de fase/DDS, sincronización con `millis()`, envolvente ADSR, MIDI).